

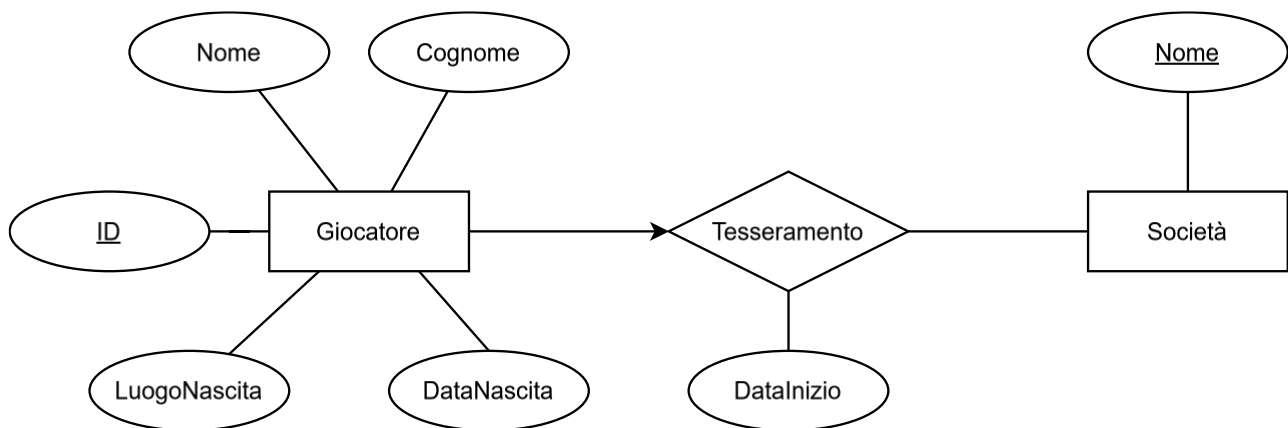
Esercitazione ER

Esercizio 1

Testo

Ogni giocatore di calcio (descritto da nome, cognome, luogo e data di nascita) può essere tesserato per una sola società sportiva. A noi interessa registrare solo la società (identificata con il nome) presso cui un giocatore è attualmente tesserato e da quando.

Modello ER



Modello relazionale

```
CREATE TABLE Societa (  
    Nome VARCHAR(100) PRIMARY KEY  
);  
  
CREATE TABLE Giocatore (  
    ID_Giocatore INT PRIMARY KEY,  
    Nome VARCHAR(50),  
    Cognome VARCHAR(50),  
    LuogoNascita VARCHAR(100),  
    DataNascita DATE,  
    NomeSocieta VARCHAR(100),  
    DataInizioTesseramento DATE,  
    FOREIGN KEY (NomeSocieta) REFERENCES Societa(Nome) – Al massimo una società  
);
```

Esercizio 2

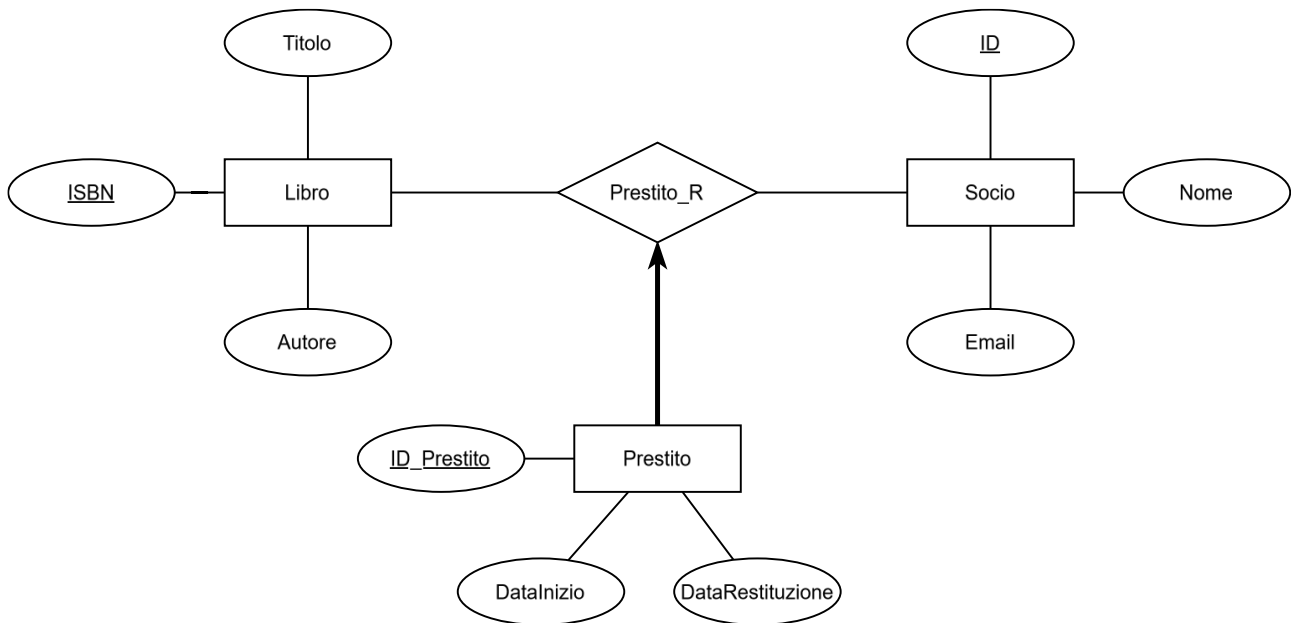
Un sistema di gestione di una biblioteca deve essere in grado di tracciare i libri prestati ai soci.

Ogni libro ha un titolo, un autore e un codice ISBN univoco.

Ogni socio ha un ID univoco, un nome e un indirizzo email.

Un socio può prendere in prestito più libri, e ogni prestito deve essere registrato con la data in cui il libro è stato preso e la data di restituzione prevista.

Modello ER



Modello relazionale

```
CREATE TABLE Socio (  
    ID_Socio INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    Email VARCHAR(100)  
);  
  
CREATE TABLE Libro (  
    ISBN VARCHAR(20) PRIMARY KEY,  
    Titolo VARCHAR(200),  
    Autore VARCHAR(100)  
);  
  
CREATE TABLE Prestito (  
    ID_Prestito INT PRIMARY KEY,  
    ID_Socio INT NOT NULL, -- Il prestito deve avere un socio  
    ISBN VARCHAR(20) NOT NULL, -- Il prestito deve avere un libro  
    Data_Prestito DATE,  
    Data_Restituzione DATE,  
    FOREIGN KEY (ID_Socio) REFERENCES Socio(ID_Socio), -- Deve avere al massimo un socio  
    FOREIGN KEY (ISBN) REFERENCES Libro(ISBN) -- Deve avere al massimo un libro  
);
```

Esercizio 3

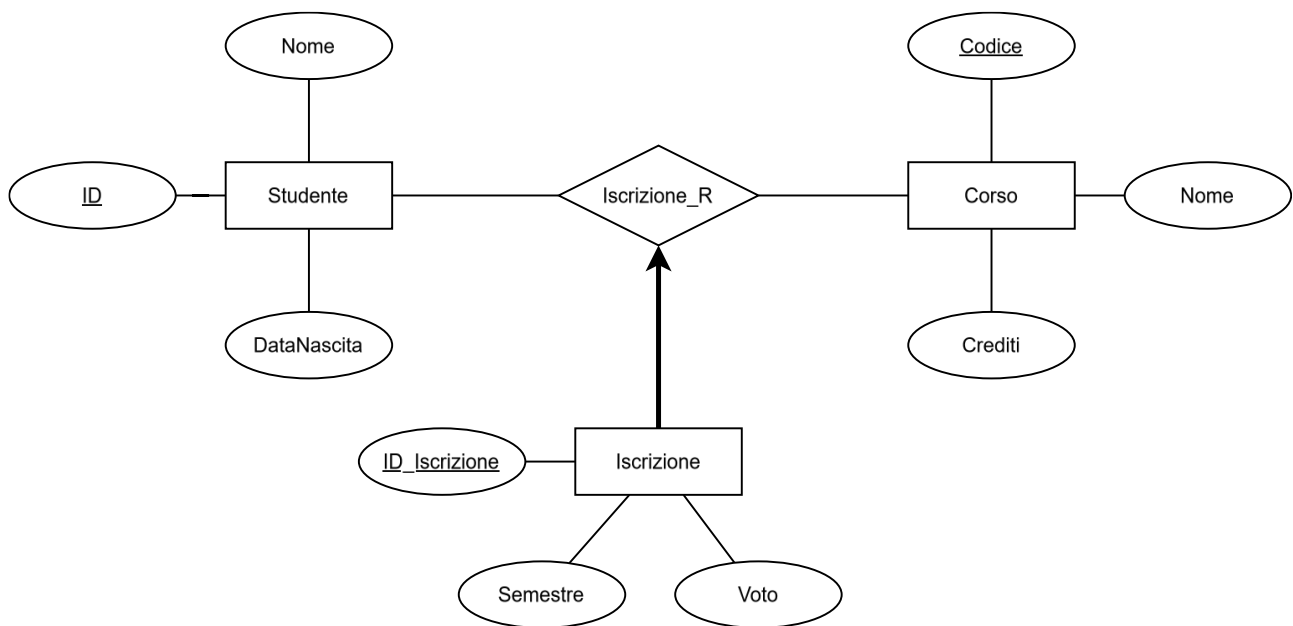
Un'università vuole un sistema per gestire i corsi e le iscrizioni degli studenti.

Ogni corso ha un codice univoco, un nome e un numero di crediti.

Gli studenti hanno un ID univoco, un nome e una data di nascita.

Ogni studente può essere iscritto a più corsi, e per ogni iscrizione si deve registrare il semestre in cui lo studente ha frequentato il corso e il voto ottenuto.

Modello ER



Modello relazionale

```
CREATE TABLE Studente (  
  ID_Studente INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  DataNascita DATE  
);
```

```
CREATE TABLE Corso (  
  Codice_Corso VARCHAR(10) PRIMARY KEY,  
  Nome VARCHAR(100),  
  Crediti INT  
);
```

```
CREATE TABLE Iscrizione (  
  ID_Iscrizione INT PRIMARY KEY,  
  ID_Studente INT NOT NULL, -- L'iscrizione deve avere uno studente  
  Codice_Corso VARCHAR(10) NOT NULL, -- L'iscrizione deve avere un corso  
  Semestre VARCHAR(10),  
  Voto INT,
```

```

FOREIGN KEY (ID_Studente) REFERENCES Studente(ID_Studente), -- Deve avere al massimo uno studente
FOREIGN KEY (Codice_Corso) REFERENCES Corso(Codice_Corso) -- Deve avere al massimo un corso
);

```

Esercizio 4

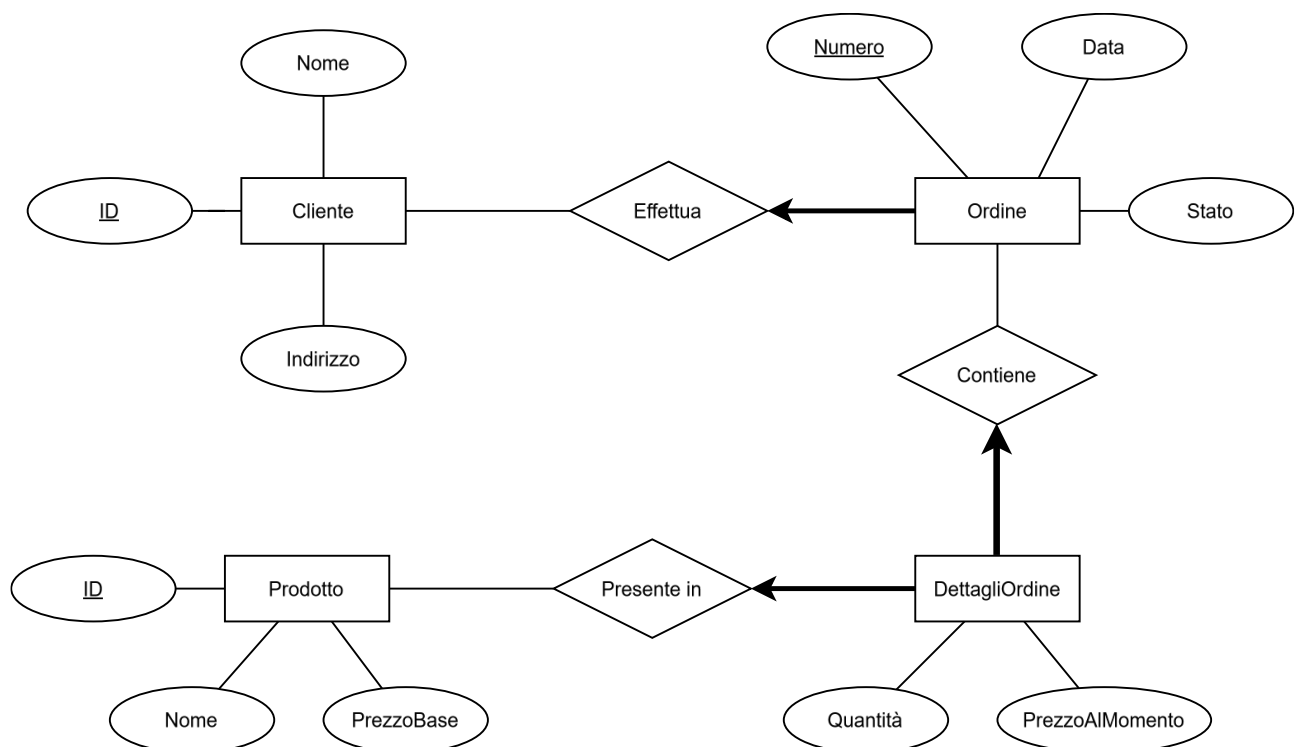
Un sito di e-commerce desidera tracciare gli ordini effettuati dai clienti.

Ogni cliente ha un ID univoco, un nome e un indirizzo.

Gli ordini hanno un numero d'ordine univoco, una data di creazione e uno stato (es. in lavorazione, spedito, consegnato).

Ogni ordine può includere più prodotti, e per ogni prodotto ordinato si deve registrare la quantità e il prezzo al momento dell'ordine.

Modello ER



Modello relazionale

```

CREATE TABLE Cliente (
  ID_Cliente INT PRIMARY KEY,
  Nome VARCHAR(100),
  Indirizzo VARCHAR(200)
);

```

```

CREATE TABLE Ordine (
  NumeroOrdine INT PRIMARY KEY,
  Data DATE,

```

```
Stato VARCHAR(50),
ID_Cliente INT NOT NULL, -- L'ordine deve avere un cliente
FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente) -- Deve avere al massimo un cliente
);
```

```
CREATE TABLE Prodotto (
    ID_Prodotto INT PRIMARY KEY,
    Nome VARCHAR(100), -- Non richiesto, ma aggiungiamo per convenienza
    PrezzoBase DECIMAL(10,2) -- Non richiesto, ma aggiungiamo per convenienza
);
```

```
CREATE TABLE DettaglioOrdine (
    NumeroOrdine INT NOT NULL, -- Il dettaglio deve avere un ordine
    ID_Prodotto INT NOT NULL, -- Il dettaglio deve avere un prodotto
    Quantità INT,
    PrezzoAlMomento DECIMAL(10,2),
    PRIMARY KEY (NumeroOrdine, ID_Prodotto),
    FOREIGN KEY (NumeroOrdine) REFERENCES Ordine(NumeroOrdine), -- Deve avere al massimo un ordine
    FOREIGN KEY (ID_Prodotto) REFERENCES Prodotto(ID_Prodotto) -- Deve avere al massimo uno prodotto
);
```

Esercizio 5

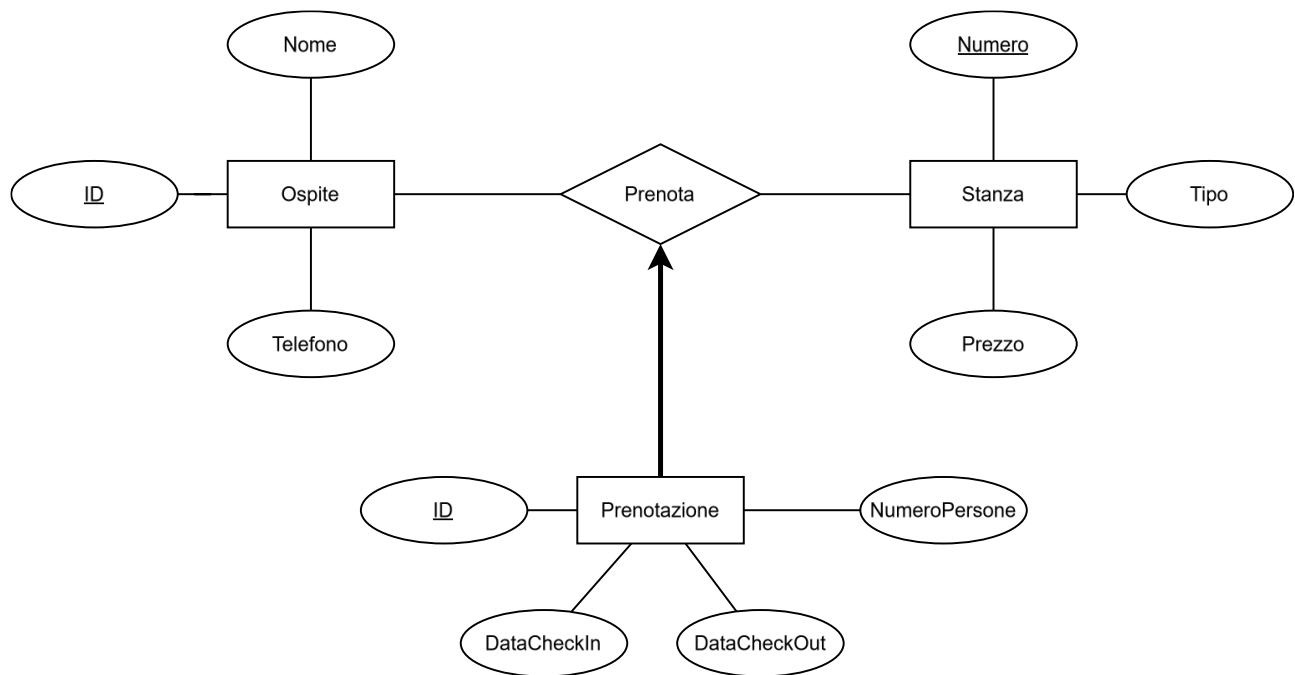
Un hotel vuole gestire le prenotazioni delle stanze.

Ogni stanza ha un numero di stanza univoco, un tipo (singola, doppia, suite) e un prezzo per notte.

Gli ospiti hanno un ID univoco, un nome e un numero di telefono.

Ogni prenotazione deve indicare l'ospite, la stanza prenotata, la data di check-in, la data di check-out e il numero di persone.

Modello ER



Modello relazionale

```
CREATE TABLE Ospite (  
    ID_Ospite INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    Telefono VARCHAR(20)  
);  
  
CREATE TABLE Stanza (  
    NumeroStanza INT PRIMARY KEY,  
    Tipo VARCHAR(50),  
    PrezzoPerNotte DECIMAL(10,2)  
);  
  
CREATE TABLE Prenotazione (  
    ID_Prenotazione INT PRIMARY KEY,  
    ID_Ospite INT NOT NULL, -- La prenotazione deve avere un ospite  
    NumeroStanza INT NOT NULL, -- La prenotazione deve avere una stanza  
    DataCheckIn DATE,  
    DataCheckOut DATE,  
    NumeroPersone INT,  
    FOREIGN KEY (ID_Ospite) REFERENCES Ospite(ID_Ospite), -- Deve avere al massimo un ospite  
    FOREIGN KEY (NumeroStanza) REFERENCES Stanza(NumeroStanza) -- Deve avere al massimo una stanza  
);
```

Esercizio 6

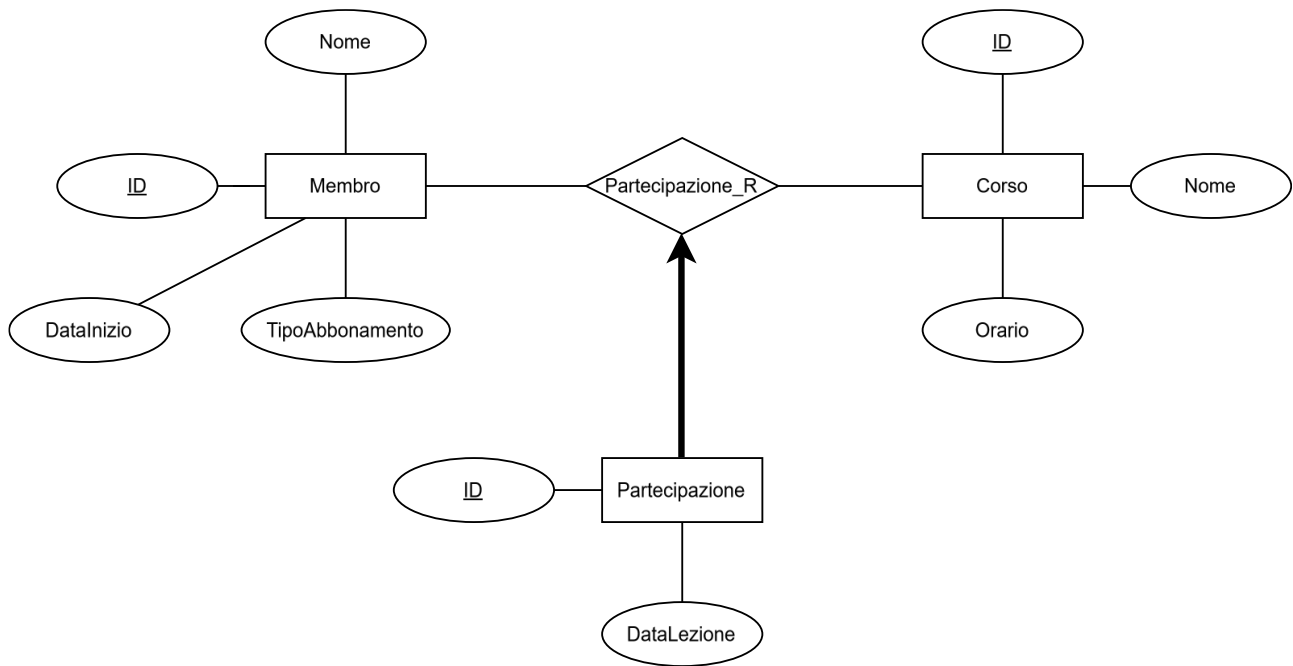
Una palestra vuole tenere traccia degli abbonamenti dei membri e dei corsi che frequentano.

Ogni membro ha un ID univoco, un nome, un tipo di abbonamento (mensile, trimestrale, annuale) e una data di inizio abbonamento.

I corsi hanno un ID univoco, un nome e un orario.

Un membro può frequentare più corsi, e per ogni partecipazione si deve registrare la data della lezione.

Modello ER



Modello relazionale

```

CREATE TABLE Membro (
  ID_Membro INT PRIMARY KEY,
  Nome VARCHAR(100),
  TipoAbbonamento VARCHAR(20),
  DataInizioAbbonamento DATE
);

```

```

CREATE TABLE Corso (
  ID_Corso INT PRIMARY KEY,
  Nome VARCHAR(100),
  Orario TIME
);

```

```

CREATE TABLE Partecipazione (
  ID_Partecipazione INT PRIMARY KEY,
  ID_Membro INT NOT NULL, -- La partecipazione deve avere un membro
  ID_Corso INT NOT NULL, -- La partecipazione deve avere un corso
  DataLezione DATE,
  FOREIGN KEY (ID_Membro) REFERENCES Membro(ID_Membro), -- Deve avere al massimo un membro
  FOREIGN KEY (ID_Corso) REFERENCES Corso(ID_Corso) -- Deve avere al massimo un corso
);

```